



AI Build Sprint – Problem Statement 1

Goal-to-Multi-Agent Orchestrator

◆ Objective

Build a no-code AI system that can take a **high-level instruction** such as:

“Analyze this insurance claim, check validity, estimate payout, and generate a customer response.”

and automatically:

- Interpret the intent
 - Decompose it into multiple tasks
 - Execute tasks through a **multi-step workflow (sequential and/or parallel)**
 - Generate a **final structured decision with reasoning and confidence**
-

◆ Expectation (Important)

Teams are expected to go **beyond a basic prototype** and demonstrate:

- Clear workflow design
 - Decision-making logic
 - Handling of multiple cases
 - Robustness and explainability
-

◆ System Requirements

Your system must include the following components:

1. Claim Analysis
2. Coverage Validation
3. Fraud / Consistency Check

4. Damage Assessment (from text and image)
 5. Payout Estimation
 6. Customer Communication
-

◆ Workflow Requirements

- Minimum **5 distinct steps**
- Must include **decision points (branching logic)**
- Workflow can be:
 - Sequential, and/or
 - Parallel (e.g., fraud check and coverage check simultaneously)

! Single prompt-based solutions are not acceptable.

◆ Input Format

Each claim will follow this structure:

Claim ID: C1

Description: Rear-end collision at signal, minor bumper damage

Policy Type: Comprehensive

Claim Amount: ₹25,000

Past Claims: 0

Documents: Complete / Missing

Image: teams may use relevant images downloaded from web

◆ Multiple Case Processing (Mandatory)

Your system must process **at least 3 claims** and generate outputs for each.

◆ Image Usage

- Teams may optionally use images (download from web)
- Use AI tools to:
 - Infer damage severity
 - Cross-check with description

Image usage is **strongly encouraged for higher evaluation**

◆ Policy Rules (Simplified)

Coverage:

- **Comprehensive Policy** → Covers own damage
 - **Third-Party Policy** → Does NOT cover own damage
-

Validity Rules:

A claim is valid if:

- Covered under policy
 - Documents are complete
 - No strong fraud indicators
-

Fraud Indicators:

- More than **3 past claims**
 - High claim amount for minor damage
 - Missing or inconsistent details
-

◆ Damage Classification

- **Minor** → scratch, dent, bumper damage
- **Moderate** → panel damage, broken lights
- **Major** → severe impact, engine damage

◆ Payout Logic

Damage Type	Payout %
Minor	70% – 90%
Moderate	50% – 70%
Major	30% – 50%

- Deductible: **₹3,000**
- Final payout = (% of claim amount) – deductible

Teams must justify the selected percentage.

◆ Confidence Score

- High (0.8–0.95) → Clear case
 - Medium (0.6–0.8) → Some ambiguity
 - Low (<0.6) → Missing data / fraud risk
-

◆ Expected Output Format

For each claim:

{

"Claim ID": "C1",

"Status": "Approved / Rejected / Pending",

```
"Estimated Payout": "₹XXXX",  
"Confidence Score": 0.85,  
"Reason": "Brief explanation",  
"Customer Message": "Clear and polite message"  
}
```

◆ **Explanation Requirement (Important)**

Your system must provide:

- Step-wise reasoning **OR**
 - A clear explanation of how the decision was made
-

◆ **Edge Case Handling (Mandatory for Full Score)**

Your system should handle:

- Fraud cases
 - Missing documents
 - Invalid policies
 - Conflicting inputs (e.g., text vs image mismatch)
-

◆ **Enhancements**

- Process more than 3 claims
 - Use images effectively in decision-making
 - Build a simple UI/dashboard
 - Use multiple AI tools
 - Provide structured logs or workflow visualization
-

◆ **Tools**

Use no-code AI tools only
